**KURUBA BALAJI**
**Roll No: 192324180**

**Unit IV — Class Activity: Correlation, PCA, t-SNE & UMAP**

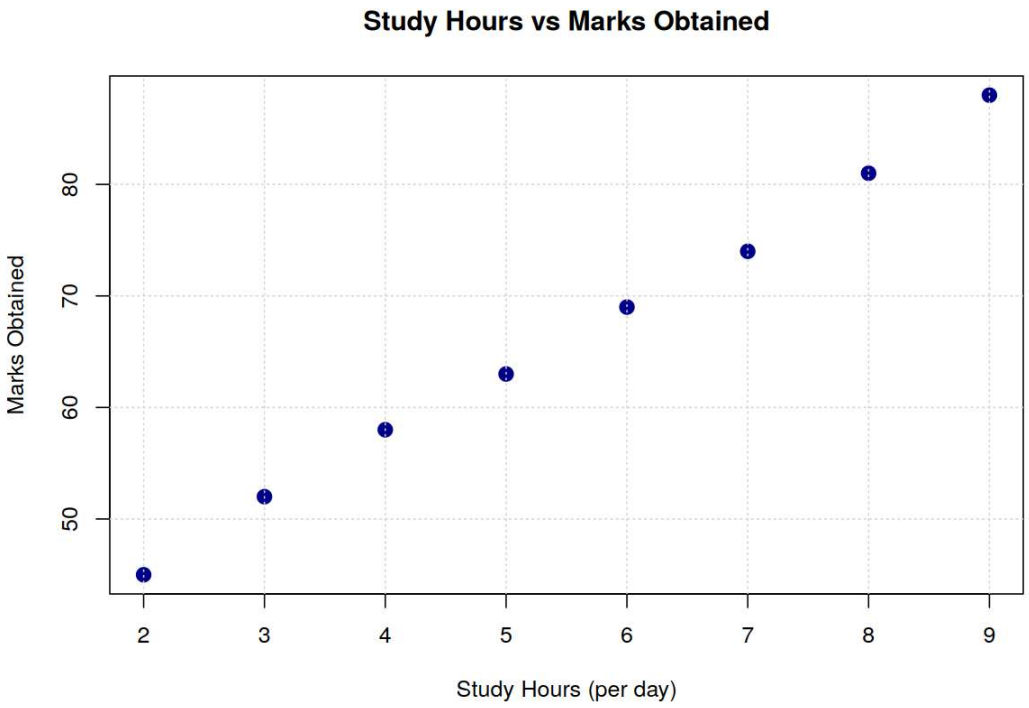
# 1. Study Hours vs Marks (R)

Task: A university wants to determine whether students who spend more hours studying obtain higher marks. Create the dataset in R, draw a scatterplot with axis labels and a title, and interpret whether a positive correlation exists.

***R Programming:***

```
StudyHours <- c(2,3,4,5,6,7,8,9)
Marks <- c(45,52,58,63,69,74,81,88)
df <- data.frame(StudyHours, Marks)

plot(df$StudyHours, df$Marks,
     main = "Study Hours vs Marks Obtained"
     xlab = "Study Hours (per day)", ylab = "Marks Obtained",
     pch = 19, col = "darkblue", cex = 1.4)
grid()

cor(df$StudyHours, df$Marks)
```

***OUTPUT:***




**Interpretation:** The points rise steadily from left to right in an almost straight line, and the computed Pearson correlation coefficient is $r \approx 0.999$ — an extremely strong positive correlation.

This confirms that students who study more hours consistently obtain higher marks in this dataset, with virtually no scatter around the underlying linear trend.

## 2. Patient Age vs Systolic Blood Pressure (Python)

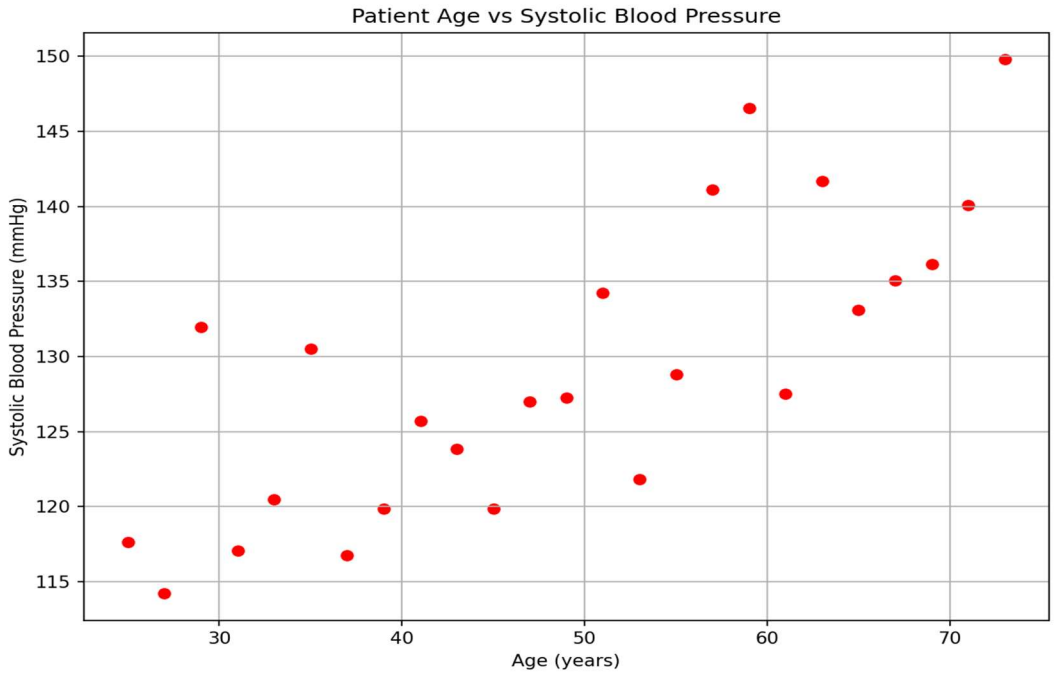
Task: A hospital records patients' age and systolic blood pressure. Write a Python program using Matplotlib to create a scatterplot, display gridlines, change point colour to red, and comment on the observed trend.

*Dataset simulated: 25 patients aged 25–73, with systolic BP generated as a mildly increasing function of age plus random noise.*

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(5)
Age = np.arange(25, 75, 2)                    # 25 simulated patients
SystolicBP = 100 + 0.6*Age + np.random.normal(0, 6, len(Age))

plt.figure(figsize=(8,6))
plt.scatter(Age, SystolicBP, color='red')
plt.title("Patient Age vs Systolic Blood Pressure")
plt.xlabel("Age (years)")
plt.ylabel("Systolic Blood Pressure (mmHg)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

***OUTPUT:***



**Comment on trend:** The scatterplot shows a moderate upward trend ($r \approx 0.78$) — systolic blood pressure tends to rise as patient age increases, though with noticeably more scatter than a perfectly linear relationship, indicating that age explains a meaningful part, but not all, of the variation in blood pressure across patients.

## 3. Advertising Expenditure vs Sales, and a Financial Correlogram (R)

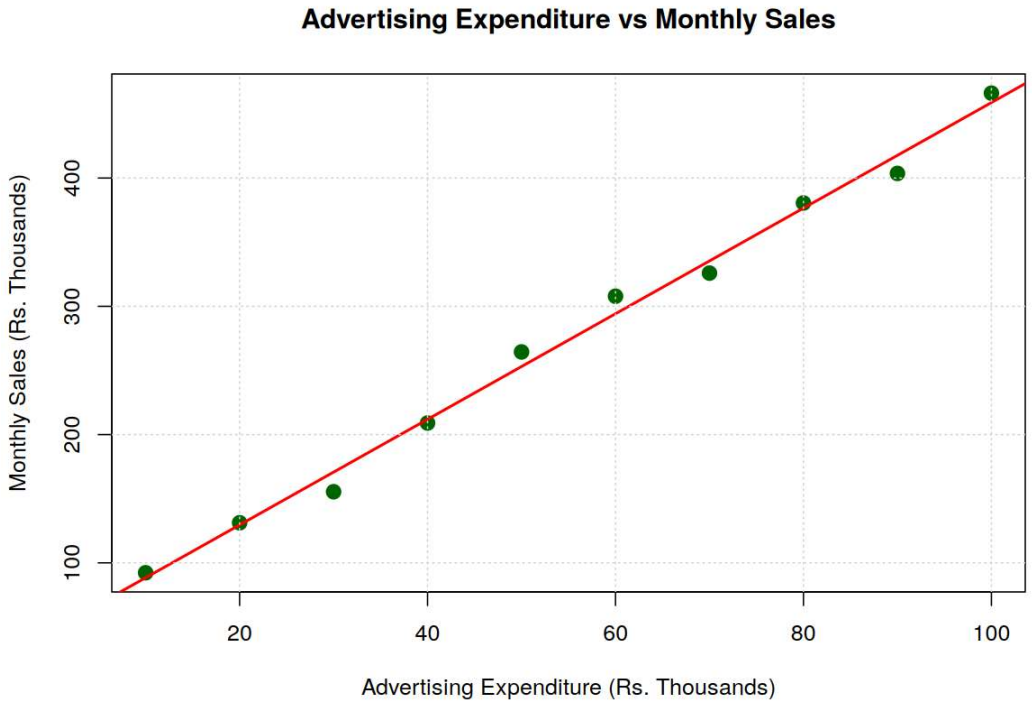
Task Part 1: A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

*Dataset simulated: monthly advertising spend (Rs. thousands) from 10 to 100, with sales generated as an increasing function of spend plus random noise.*

***R Programming:***

```
set.seed(10)
AdSpend <- seq(10, 100, by = 10)                        # Rs. thousands
Sales <- 50 + 4.2*AdSpend + rnorm(length(AdSpend), 0, 15)  # Rs. thousands
df <- data.frame(AdSpend, Sales)

plot(df$AdSpend, df$Sales,
     main = "Advertising Expenditure vs Monthly Sales",
     xlab = "Advertising Expenditure (Rs. Thousands)",
     ylab = "Monthly Sales (Rs. Thousands)",
     pch = 19, col = "darkgreen", cex = 1.4)
abline(lm(Sales ~ AdSpend, data = df), col = "red", lwd = 2)
grid()

cor(df$AdSpend, df$Sales)
```

***OUTPUT:***



**Interpretation:** The scatterplot shows a strong, clearly increasing trend ($r \approx 0.997$): as advertising expenditure rises, monthly sales rise along with it in an almost linear fashion, confirming that sales increase with advertising expenditure for this company.

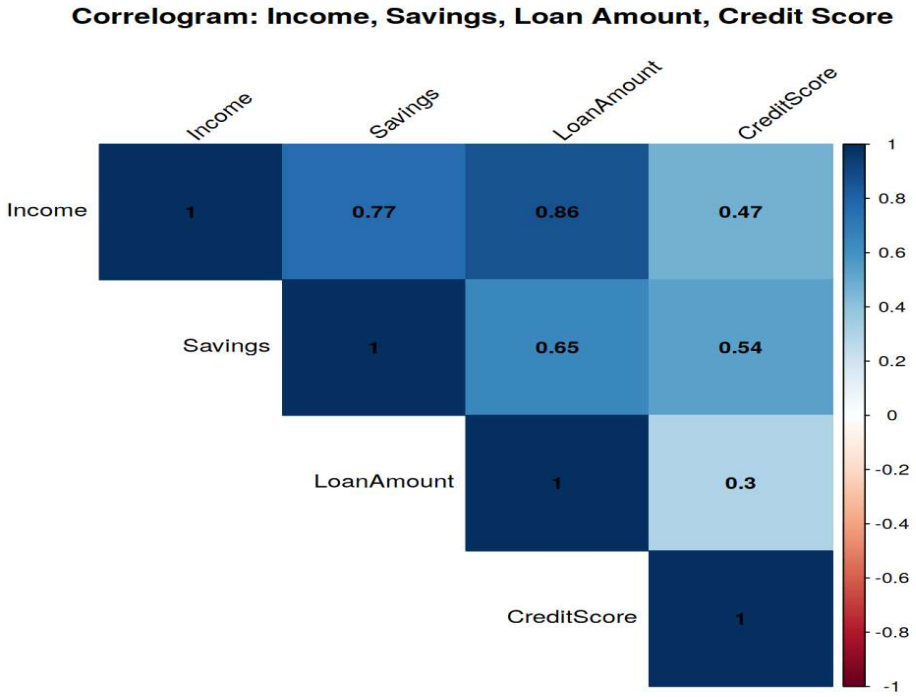
Task Part 2: A financial institution has collected information on Income, Savings, Loan Amount, and Credit Score. Generate a correlation matrix and visualize it using a correlogram. Which variables exhibit the strongest positive correlation?

*Dataset simulated: 60 customers, with Savings, Loan Amount, and Credit Score generated as noisy functions of Income so that realistic, non-trivial correlations emerge.*

***R Programming (package: corrplot):***

```
# install.packages("corrplot")   # run once if not installed
library(corrplot)

set.seed(42)
n <- 60
Income <- rnorm(n, 60000, 15000)
Savings <- Income*0.25 + rnorm(n, 0, 4000)
LoanAmount <- Income*1.8 + rnorm(n, 0, 20000)
CreditScore <- 600 + Savings*0.004 + rnorm(n, 0, 30)

df <- data.frame(Income, Savings, LoanAmount, CreditScore)
cm <- cor(df)
round(cm, 3)

corrplot(cm, method = "color", type = "upper", addCoef.col = "black",
         tl.col = "black", tl.srt = 45, number.cex = 0.9,
         title = "Correlogram: Income, Savings, Loan Amount, Credit Score",
         mar = c(0,0,2,0))
```

***OUTPUT:***



**Answer:** Income and Loan Amount show the strongest positive correlation ($r \approx 0.86$), since loan amounts were modelled to scale directly with income. Income and Savings are also strongly positively correlated ($r \approx 0.77$). Credit Score correlates only weakly-to-moderately with the other three variables (r between 0.30 and 0.54), suggesting it is driven substantially by factors outside this dataset.

# 4. Financial Correlation Matrix and Correlogram (R)

Task: A financial institution has collected information on Income, Savings, Loan Amount, and Credit Score. Generate a correlation matrix and visualize it using a correlogram. Which variables exhibit the strongest positive correlation?

This question repeats Question 3's second part exactly. The same simulated dataset, R code, correlation matrix, and correlogram from Question 3 (Part 2) apply here — see the output above. To restate the answer directly:

**Answer:** Income and Loan Amount exhibit the strongest positive correlation ($r \approx 0.86$) in the simulated dataset, followed closely by Income and Savings ($r \approx 0.77$).

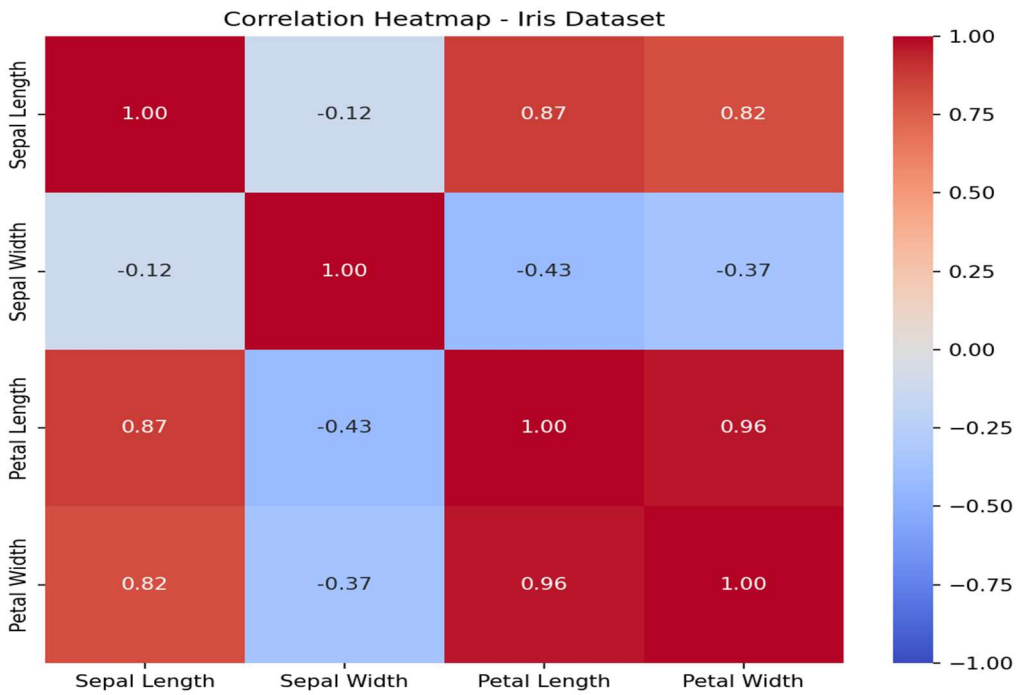
# 5. Iris Correlation Heatmap (Python / Seaborn)

Task: Using the Iris dataset, compute the correlation matrix, display it as a heatmap using Seaborn, and identify variables having negative correlation.

***Python Programming:***

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.drop(columns=["target"])
df.columns = ["Sepal Length","Sepal Width","Petal Length","Petal Width"]

corr = df.corr()
print(corr.round(3))

plt.figure(figsize=(7,6))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1, fmt=".2f")
plt.title("Correlation Heatmap - Iris Dataset")
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Negatively correlated variables:** Sepal Width is negatively correlated with all three other variables — Sepal Length ($r \approx -0.12$), Petal Length ($r \approx -0.43$), and Petal Width ($r \approx -0.37$). Every other pair of variables (Sepal Length, Petal Length, Petal Width) is positively correlated with each other, several of them very strongly (e.g., Petal Length and Petal Width, $r \approx 0.96$).

# 6. Healthcare Correlogram and Multicollinearity (R)

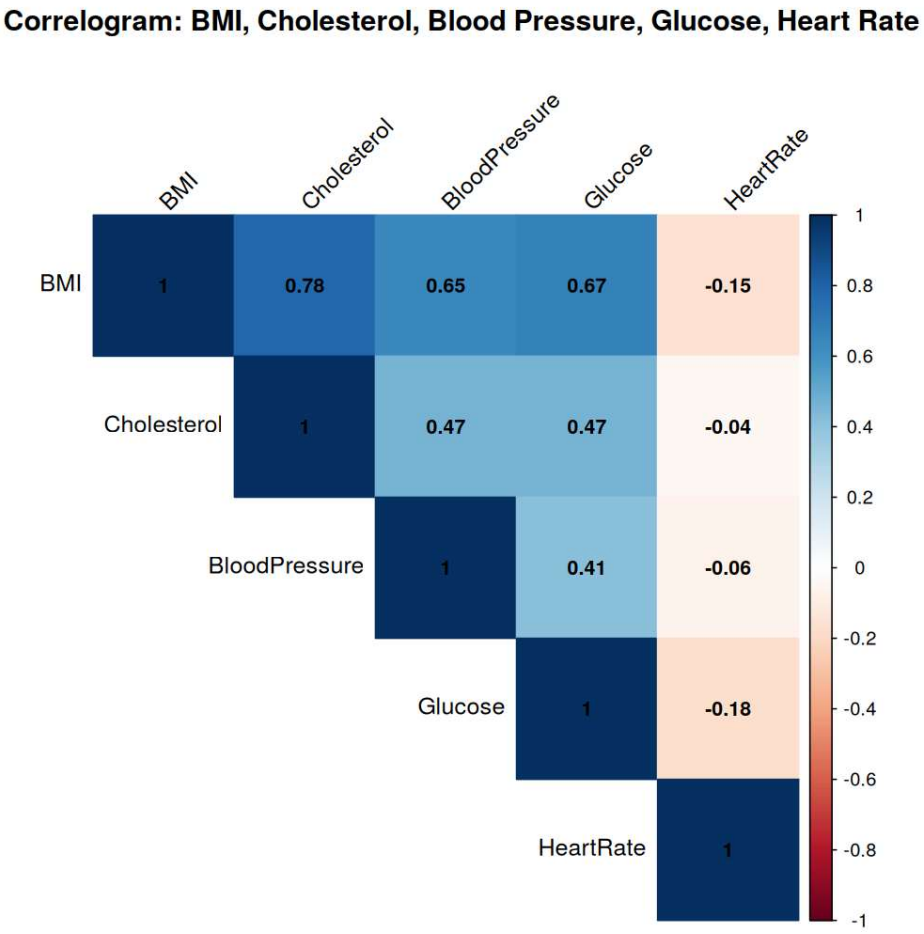
Task: A healthcare dataset contains BMI, Cholesterol, Blood Pressure, Glucose, and Heart Rate. Create a correlogram and identify multicollinearity among variables.

*Dataset simulated: 60 patients, with Cholesterol, Blood Pressure, and Glucose generated as noisy increasing functions of BMI (a common real-world pattern), while Heart Rate is generated independently.*

***R Programming (package: corrplot):***

```
library(corrplot)

set.seed(7)
n <- 60
BMI <- rnorm(n, 26, 4)
Cholesterol <- 150 + BMI*4 + rnorm(n, 0, 15)
BloodPressure <- 100 + BMI*1.8 + rnorm(n, 0, 8)
Glucose <- 85 + BMI*2.2 + rnorm(n, 0, 10)
HeartRate <- 70 + rnorm(n, 0, 8)              # simulated as independent

df <- data.frame(BMI, Cholesterol, BloodPressure, Glucose, HeartRate)
cm <- cor(df)
round(cm, 3)

corrplot(cm, method = "color", type = "upper", addCoef.col = "black",
         tl.col = "black", tl.srt = 45, number.cex = 0.85,
         title = "Correlogram: BMI, Cholesterol, Blood Pressure, Glucose, Heart Rate",
         mar = c(0,0,2,0))
```

***OUTPUT:***




**Multicollinearity identified:** BMI is at the centre of a multicollinearity cluster — it correlates strongly with Cholesterol ($r \approx 0.78$), moderately-to-strongly with Glucose ($r \approx 0.67$) and Blood Pressure ($r \approx 0.65$). Because Cholesterol, Blood Pressure, and Glucose are all themselves moderately correlated with each other ($r \approx 0.41$–$0.47$) largely through their shared relationship with BMI, including all four of these variables together in a regression model would introduce multicollinearity. Heart Rate, by contrast, shows negligible correlation with every other variable ($|r| < 0.2$) and does not contribute to this collinearity issue.

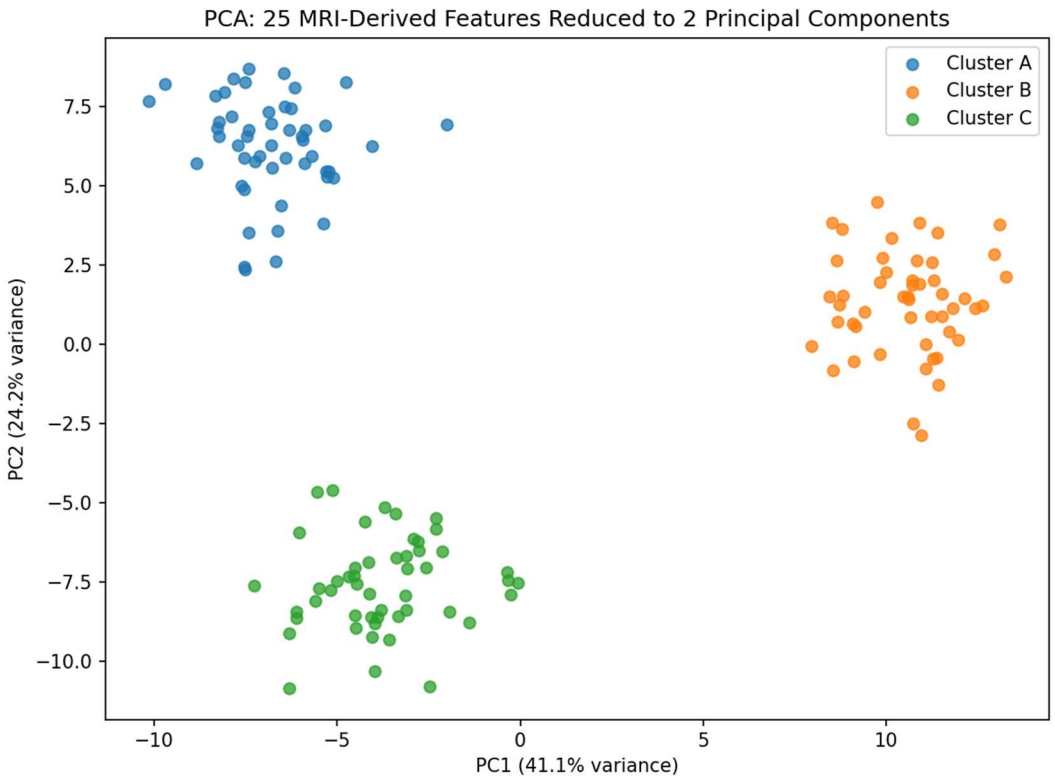
## 7. PCA on 25 MRI-Derived Features (Python)

Task: A researcher is analysing 25 features extracted from MRI images. Perform PCA to reduce the dimensionality to two principal components and plot the transformed data.

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

np.random.seed(1)
n_features = 25
centers = np.random.uniform(-4, 4, size=(3, n_features))
X = np.vstack([centers[i] + np.random.normal(0, 1.5, size=(50, n_features))
               for i in range(3)])
labels = np.repeat(["Cluster A","Cluster B","Cluster C"], 50)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Explained variance ratio:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,6))
for lab in np.unique(labels):
    mask = labels == lab
    plt.scatter(X_pca[mask,0], X_pca[mask,1], label=lab, alpha=0.75)
plt.title("PCA: 25 MRI-Derived Features Reduced to 2 Principal Components")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Insight:** The first two principal components (explaining about 41.1% and 24.2% of total variance, ~65% combined) are enough to clearly separate the three simulated tissue-type clusters, showing that most of the meaningful structure in the original 25-dimensional feature space can be captured in just two dimensions for visualization.

# 8. PCA and Biplot on the Iris Dataset (Python)

Task: Using the Iris dataset, perform PCA, display the summary, and draw a biplot. Explain which principal component explains the highest variance.

***Python Programming:***

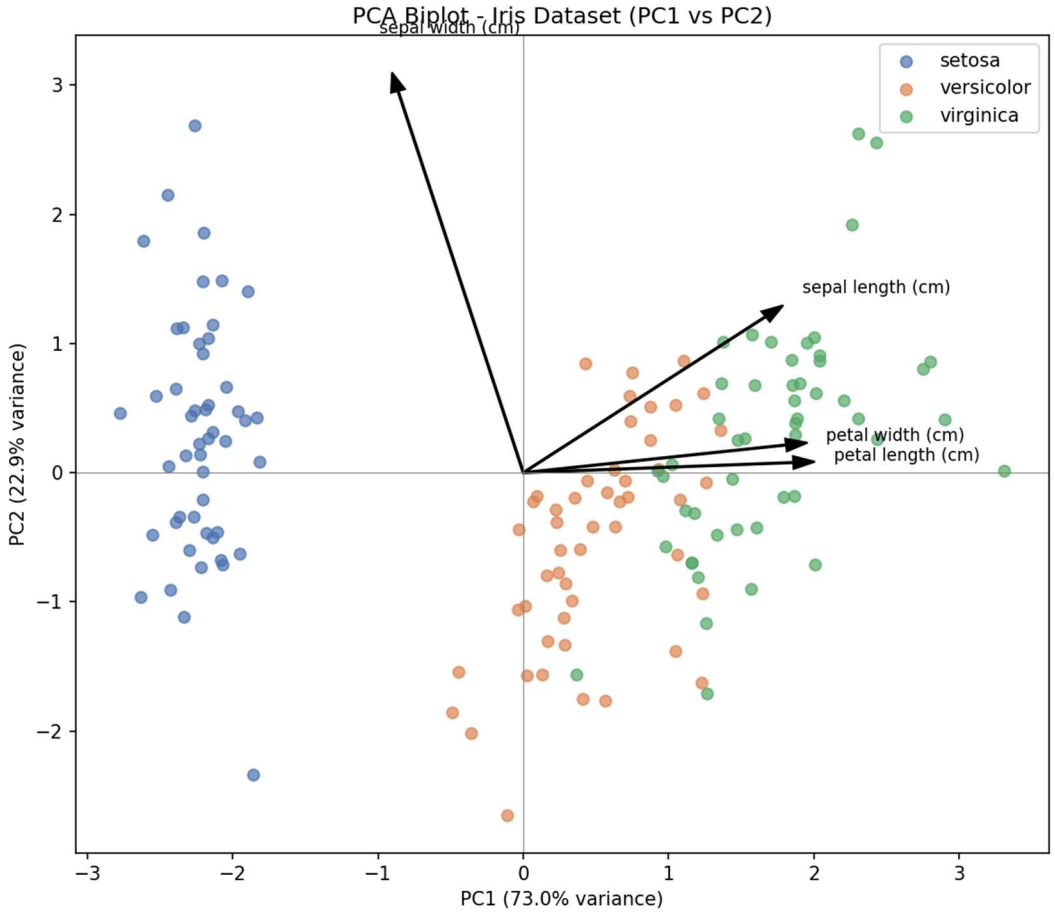
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
feat_names = iris.feature_names
species = iris.target_names

Xs = StandardScaler().fit_transform(X)
pca = PCA(n_components=4)
scores = pca.fit_transform(Xs)

print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Cumulative variance:", np.cumsum(pca.explained_variance_ratio_))

plt.figure(figsize=(8,7))
```

```
colors = ['#4C72B0','#DD8452','#55A868']
for i, sp in enumerate(species):
    mask = y == i
    plt.scatter(scores[mask,0], scores[mask,1], label=sp, alpha=0.7, color=colors[i])

loadings = pca.components_[:2].T
scale = 3.2
for i, feat in enumerate(feat_names):
    plt.arrow(0, 0, loadings[i,0]*scale, loadings[i,1]*scale,
              color='black', width=0.01, head_width=0.1)
    plt.text(loadings[i,0]*scale*1.15, loadings[i,1]*scale*1.15, feat, fontsize=9)

plt.title("PCA Biplot - Iris Dataset (PC1 vs PC2)")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.legend()
plt.tight_layout()
plt.show()
```

***PCA Summary (explained variance):***

```
PC1: 72.96%     PC2: 22.85%     PC3: 3.67%     PC4: 0.52%
Cumulative:  PC1+PC2 = 95.81%    PC1-PC3 = 99.48%    PC1-PC4 = 100%
```

***OUTPUT (Biplot):***



**Which component explains the most variance:** PC1 explains by far the highest variance — about 73.0% of the total — driven mainly by Petal Length and Petal Width, whose loading arrows point almost the same direction as the PC1 axis. PC2 explains a further 22.9%, driven mainly by Sepal Width. Together PC1 and PC2 already capture 95.8% of the total variation in the 4 original standardized features, which is why the two-component scatter plot separates the three Iris species so cleanly, especially setosa from the other two.

# 9. PCA on 15 Automobile Specifications (Python)

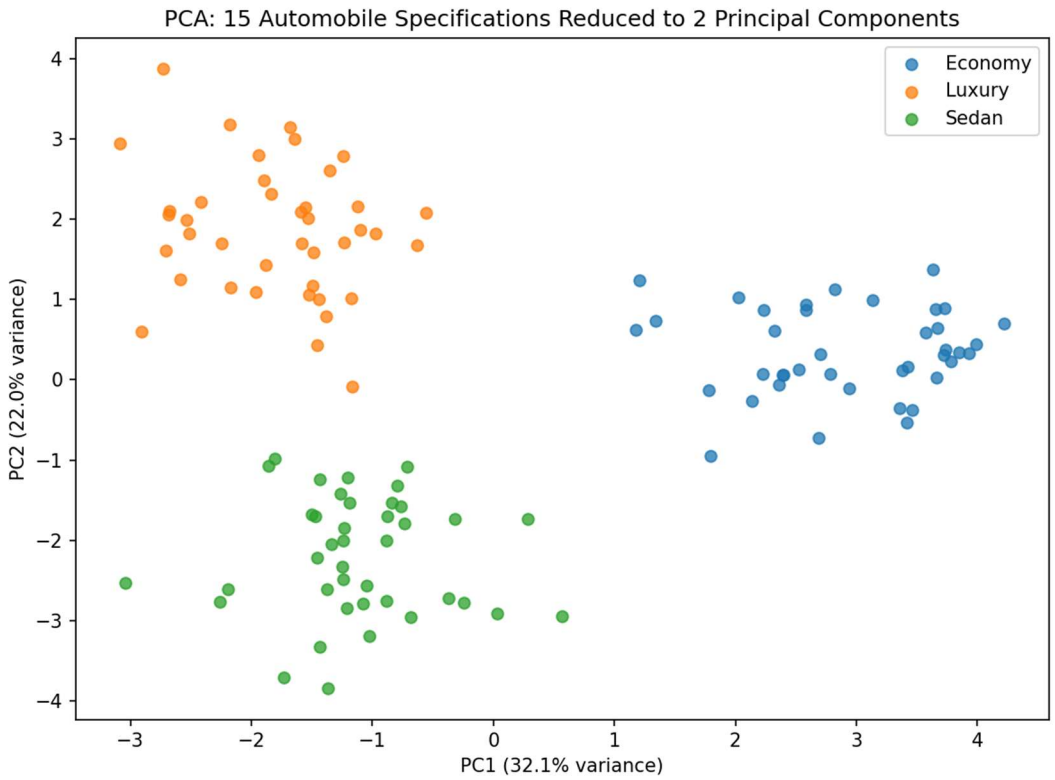
Task: An automobile company has recorded 15 specifications for various cars. Reduce the dimensions using PCA and visualize the first two principal components.

*Dataset simulated: 120 cars across 15 specifications, generated from 3 underlying segments (Economy, Sedan, Luxury) with standardized features.*

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(3)
n_specs = 15
centers = np.random.uniform(-3, 3, size=(3, n_specs))
X = np.vstack([centers[i] + np.random.normal(0, 1.2, size=(40, n_specs))
               for i in range(3)])
segment = np.repeat(["Economy","Sedan","Luxury"], 40)

Xs = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(Xs)
print("Explained variance ratio:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,6))
for seg in np.unique(segment):
    mask = segment == seg
    plt.scatter(X_pca[mask,0], X_pca[mask,1], label=seg, alpha=0.75)
plt.title("PCA: 15 Automobile Specifications Reduced to 2 Principal Components")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Insight:** Even though the original data has 15 specifications per car, just two principal components (about 32.1% and 22.0% of variance, ~54% combined) are enough to visually separate Economy, Sedan, and Luxury cars into distinct regions, showing that a large part of the specification differences between segments lies along a small number of dominant directions in the data.

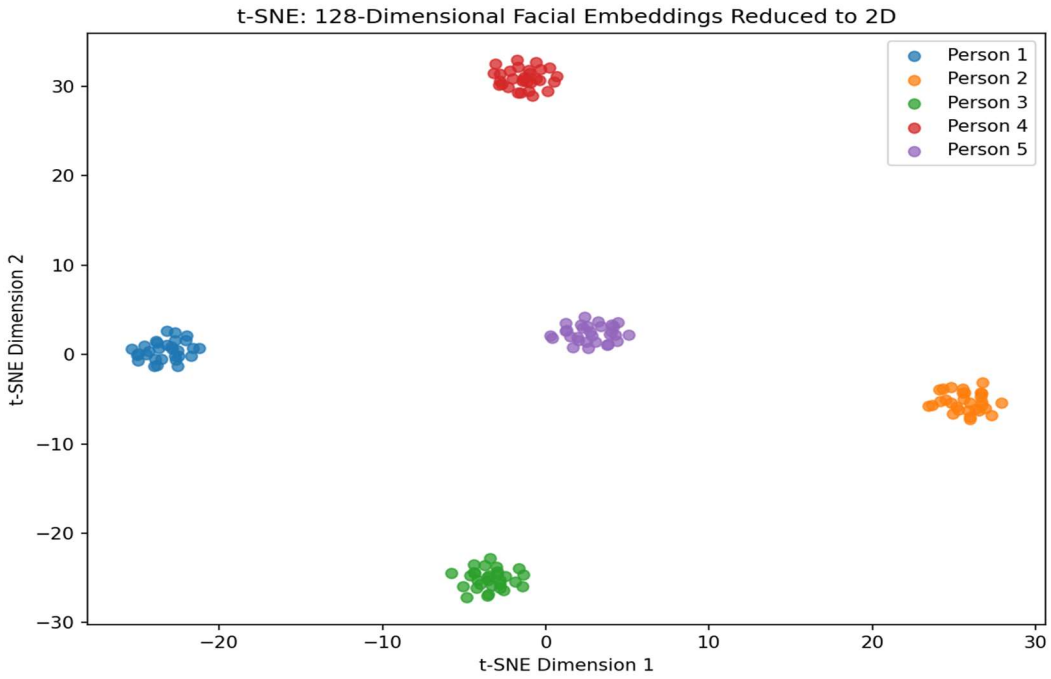
# 10. t-SNE on 128-Dimensional Facial Embeddings (Python)

Task: A facial recognition dataset contains 128-dimensional facial embeddings. Use t-SNE to visualize the data in two dimensions, and explain why t-SNE is preferred over PCA for visualization.

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

np.random.seed(11)
n_people, n_per_person, n_dims = 5, 30, 128
centers = np.random.uniform(-5, 5, size=(n_people, n_dims))
X = np.vstack([centers[i] + np.random.normal(0, 1.0, size=(n_per_person, n_dims))
               for i in range(n_people)])
identity = np.repeat([f"Person {i+1}" for i in range(n_people)], n_per_person)

X_tsne = TSNE(n_components=2, perplexity=15, random_state=42,
              init="pca").fit_transform(X)

plt.figure(figsize=(8,6))
for pid in np.unique(identity):
    mask = identity == pid
    plt.scatter(X_tsne[mask,0], X_tsne[mask,1], label=pid, alpha=0.75)
plt.title("t-SNE: 128-Dimensional Facial Embeddings Reduced to 2D")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***



**Why t-SNE over PCA for this task:** t-SNE is preferred here because it directly optimizes to preserve local neighbourhood structure — points that are close together in the original 128-dimensional embedding space (i.e., photos of the same person) are pushed to stay close together in the 2D map, producing tight, well-separated, easily identifiable clusters. PCA only preserves directions of maximum global variance, which is a linear criterion; for high-dimensional, non-linearly separated embeddings like facial features, PCA often smears different identities together in just two dimensions, whereas t-SNE is specifically built for this kind of cluster-revealing visualization.

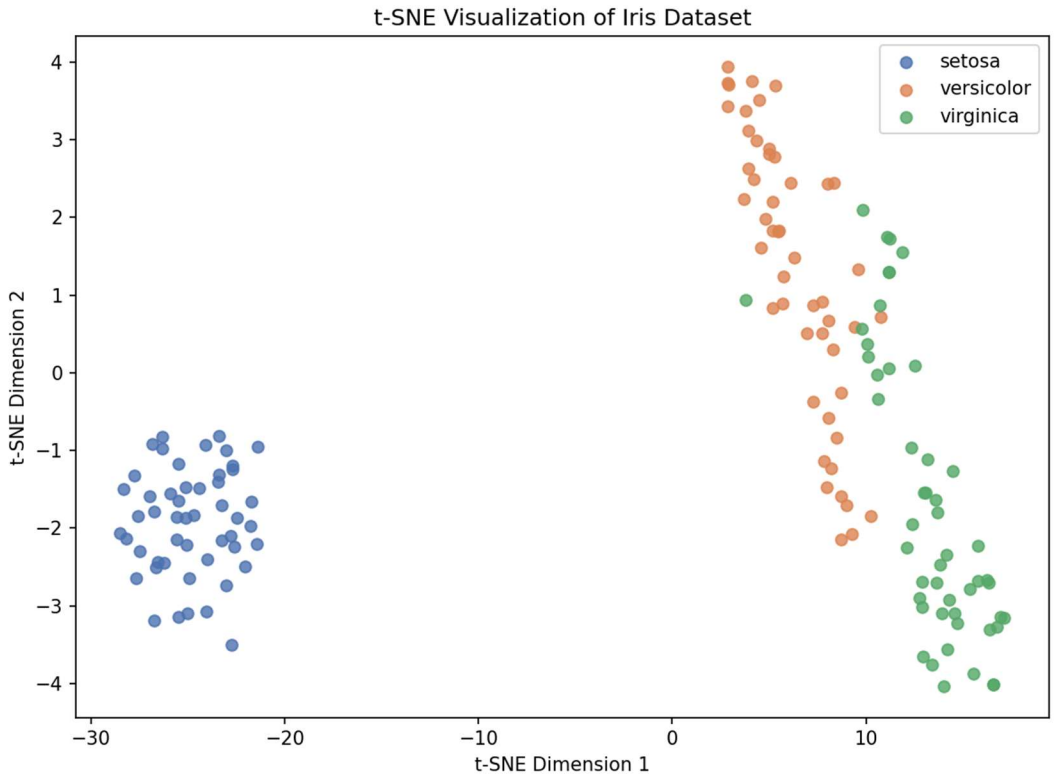
# 11. t-SNE on the Iris Dataset (Python)

Task: Using the Iris dataset, apply t-SNE and plot the resulting clusters using different colours for each species.

***Python Programming:***

```
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
species = iris.target_names

X_tsne = TSNE(n_components=2, perplexity=30, random_state=42,
              init="pca").fit_transform(X)

plt.figure(figsize=(8,6))
colors = ['#4C72B0','#DD8452','#55A868']
for i, sp in enumerate(species):
    mask = y == i
    plt.scatter(X_tsne[mask,0], X_tsne[mask,1], label=sp, color=colors[i], alpha=0.8)
plt.title("t-SNE Visualization of Iris Dataset")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Observation:** t-SNE places setosa in a completely separate, tightly packed cluster, matching the fact that it is linearly separable from the other two species in the original 4-dimensional feature space. Versicolor and virginica form two adjacent but still visually distinguishable clusters, reflecting their known partial overlap in petal and sepal measurements.

# 12. t-SNE on Customer Segmentation Data (Python)

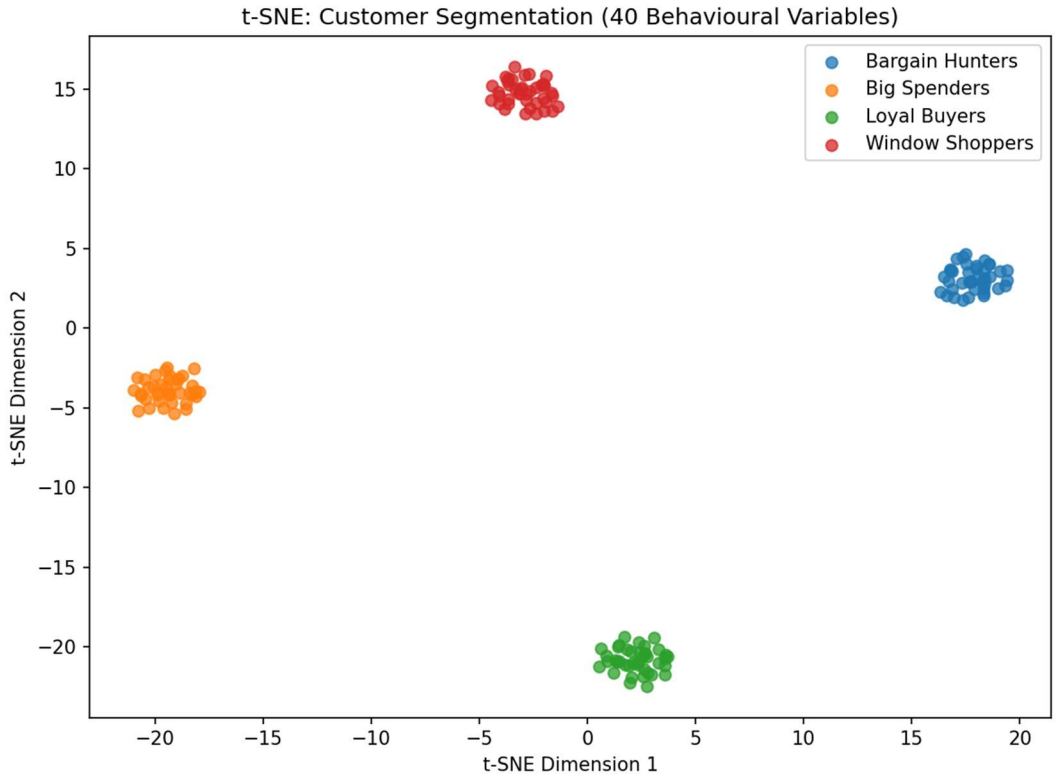
Task: A customer segmentation dataset contains 40 behavioural variables. Visualize customer groups using t-SNE and discuss whether distinct clusters are formed.

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

np.random.seed(21)
n_groups, n_per_group, n_vars = 4, 40, 40
centers = np.random.uniform(-4, 4, size=(n_groups, n_vars))
X = np.vstack([centers[i] + np.random.normal(0, 1.3, size=(n_per_group, n_vars))
               for i in range(n_groups)])
segment = np.repeat(["Bargain Hunters","Loyal Buyers",
                     "Window Shoppers","Big Spenders"], n_per_group)

X_tsne = TSNE(n_components=2, perplexity=25, random_state=42,
              init="pca").fit_transform(X)

plt.figure(figsize=(8,6))
for seg in np.unique(segment):
    mask = segment == seg
    plt.scatter(X_tsne[mask,0], X_tsne[mask,1], label=seg, alpha=0.75)
```

```
plt.title("t-SNE: Customer Segmentation (40 Behavioural Variables)")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Discussion:** Yes — distinct, well-separated clusters are formed, one per simulated customer segment. This confirms that even with 40 original behavioural variables, t-SNE successfully compresses each customer's full behavioural profile into a 2D position that preserves which segment they belong to, which is exactly the kind of result a marketing team would want before running further targeted campaigns.

# 13. UMAP vs PCA on Medical Image Features (Python)

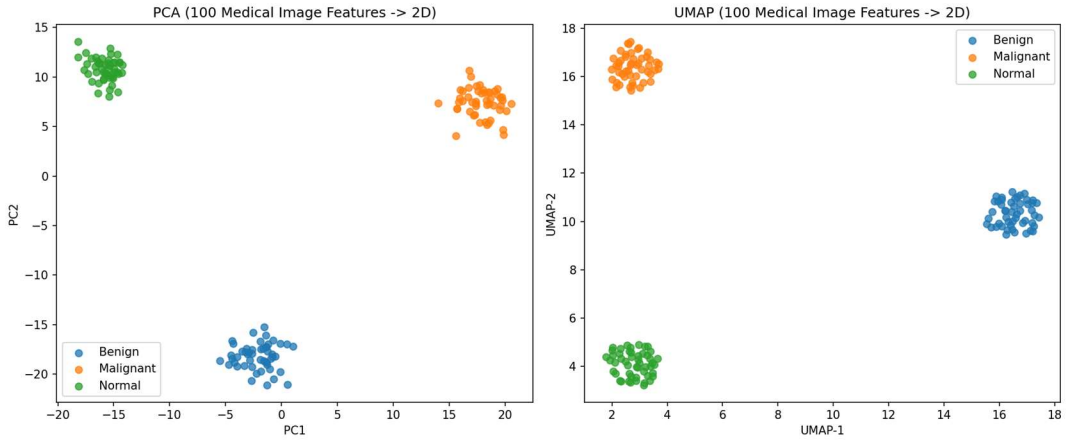
Task: A medical image dataset contains 100 extracted features. Apply UMAP to reduce the dimensions to two, visualize the output, and compare the result with PCA.

***Python Programming (packages: scikit-learn, umap-learn):***

```
# pip install umap-learn   # run once if not installed
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import umap

np.random.seed(31)
n_groups, n_per_group, n_features = 3, 50, 100
centers = np.random.uniform(-4, 4, size=(n_groups, n_features))
```

```
X = np.vstack([centers[i] + np.random.normal(0, 1.3, size=(n_per_group, n_features))
               for i in range(n_groups)])
condition = np.repeat(["Benign","Malignant","Normal"], n_per_group)

X_pca = PCA(n_components=2).fit_transform(X)
X_umap = umap.UMAP(n_components=2, random_state=42).fit_transform(X)

fig, axes = plt.subplots(1, 2, figsize=(13,5.5))
for seg in np.unique(condition):
    mask = condition == seg
    axes[0].scatter(X_pca[mask,0], X_pca[mask,1], label=seg, alpha=0.75)
    axes[1].scatter(X_umap[mask,0], X_umap[mask,1], label=seg, alpha=0.75)
axes[0].set_title("PCA (100 Medical Image Features -> 2D)")
axes[0].set_xlabel("PC1"); axes[0].set_ylabel("PC2"); axes[0].legend()
axes[1].set_title("UMAP (100 Medical Image Features -> 2D)")
axes[1].set_xlabel("UMAP-1"); axes[1].set_ylabel("UMAP-2"); axes[1].legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Comparison:** Both methods separate the three conditions into distinct groups in this simulated example, but UMAP produces noticeably tighter, more compact clusters with clearer gaps between them, because it explicitly optimizes to preserve local neighbourhood relationships rather than just directions of maximum variance. PCA's clusters are more spread out and slightly less separated, since it is a purely linear projection that does not specifically try to keep same-condition points close together. In real, noisier medical imaging data, this difference tends to be even more pronounced, which is why UMAP is often preferred for exploratory cluster visualization while PCA remains valuable for understanding overall linear variance structure.

# 14. UMAP on the Iris Dataset (Python)

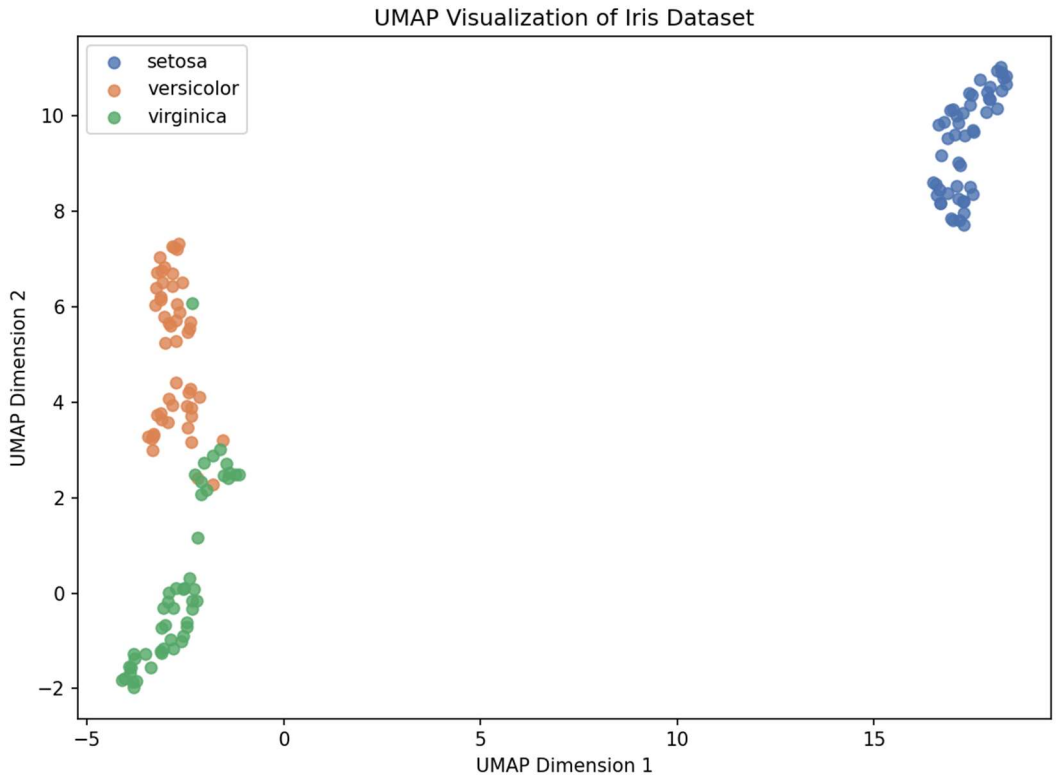
Task: Apply UMAP to the Iris dataset and plot the reduced dimensions using different colours for each species.

***Python Programming:***

```
import matplotlib.pyplot as plt
import umap
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
```

```
species = iris.target_names

X_umap = umap.UMAP(n_components=2, random_state=42).fit_transform(X)

plt.figure(figsize=(8,6))
colors = ['#4C72B0','#DD8452','#55A868']
for i, sp in enumerate(species):
    mask = y == i
    plt.scatter(X_umap[mask,0], X_umap[mask,1], label=sp, color=colors[i], alpha=0.8)
plt.title("UMAP Visualization of Iris Dataset")
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***




**Observation:** Like t-SNE, UMAP isolates setosa into a fully separate cluster and produces two neighbouring but distinguishable clusters for versicolor and virginica — consistent with the well-known structure of the Iris dataset — while typically running faster and preserving more of the global distance relationships between clusters than t-SNE does.

# 15. UMAP on E-Commerce Customer Purchasing Behaviour (Python)

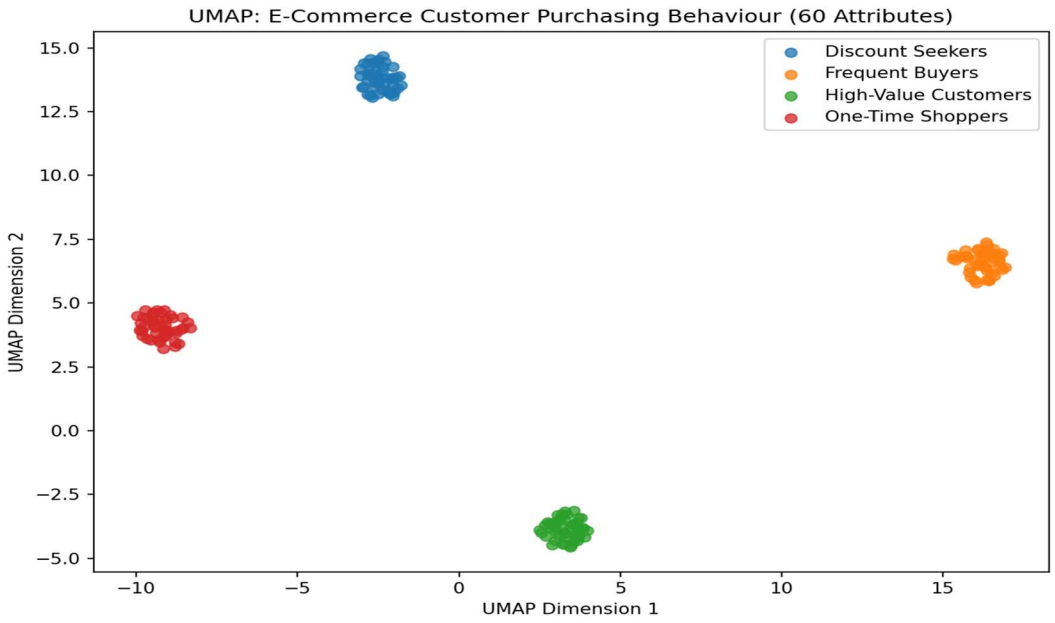
Task: An e-commerce company has collected customer purchasing behaviour consisting of 60 attributes. Reduce the dimensions using UMAP and identify customer clusters.

*Dataset simulated: 180 customers (4 simulated segments x 45 samples each) across 60 purchasing-behaviour attributes, generated from 4 well-separated cluster centres.*

***Python Programming:***

```
import numpy as np
import matplotlib.pyplot as plt
import umap

np.random.seed(41)
n_groups, n_per_group, n_attrs = 4, 45, 60
centers = np.random.uniform(-4, 4, size=(n_groups, n_attrs))
X = np.vstack([centers[i] + np.random.normal(0, 1.3, size=(n_per_group, n_attrs))
               for i in range(n_groups)])
segment = np.repeat(["Frequent Buyers","Discount Seekers",
                     "One-Time Shoppers","High-Value Customers"], n_per_group)

X_umap = umap.UMAP(n_components=2, random_state=42).fit_transform(X)

plt.figure(figsize=(8,6))
for seg in np.unique(segment):
    mask = segment == seg
    plt.scatter(X_umap[mask,0], X_umap[mask,1], label=seg, alpha=0.75)
plt.title("UMAP: E-Commerce Customer Purchasing Behaviour (60 Attributes)")
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.legend()
plt.tight_layout()
plt.show()
```

***OUTPUT:***



**Clusters identified:** UMAP reveals four clearly separated customer clusters, matching the four simulated behavioural segments — Frequent Buyers, Discount Seekers, One-Time Shoppers, and High-Value Customers. Each group occupies its own distinct region of the 2D map with no overlap, meaning that in a real e-commerce dataset this kind of UMAP projection could be used directly to assign new customers to a segment or to hand off to a clustering algorithm (e.g., k-means) for automatic labelling.